Sequence-to-Sequence Models: Intro to Attention

Naeemullah Khan
naeemullah.khan@kaust.edu.sa

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

July 16, 2026

Table of Contents

Motivation

Why Do We Need Seq2Seq and Attention?

- Many real-world problems require transforming one sequence to another:
  - Translation: `"Bonjour"` → `"Hello"`
  - Dialogue systems: Question → Response
  - Speech: Audio → Text

Standard RNNs struggle with input/output sequences of different lengths and long-term dependencies.

Seq2Seq models + attention solve this with a powerful encoder-decoder framework.

Learning Outcomes

By the end of this session, you should be able to:

- Explain the Seq2Seq architecture and encoder-decoder framework
- Understand the bottleneck problem in fixed-size representations
- Describe the motivation for and core idea behind attention mechanisms
- Appreciate how attention improves performance in NLP tasks
- Recognize future directions in attention-based modeling

Seq2Seq Architecture

Seq2Seq Architecture

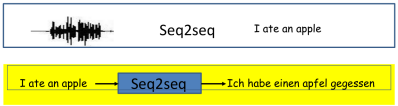
Key Idea: Map input sequence $\rightarrow$ intermediate vector $\rightarrow$ output sequence.

- **Encoder RNN:** Processes input sequence and compresses it into a **fixed-length vector** (context).
- **Decoder RNN:** Generates output sequence from the context vector.

Applications:

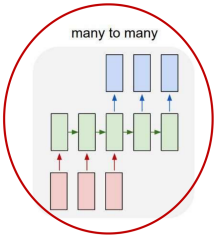
- Machine translation
- Summarization
- Dialogue systems
- Speech recognition

Seq2Seq Architecture



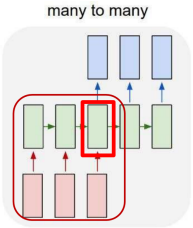
- Sequence goes in, sequence comes out
- No notion of "time synchrony" between input and output
  - May even not even maintain order of symbols
    - E.g. "I ate an apple" → "Ich habe einen apfel gegessen"
  - Or even seem related to the input
    - E.g. "My screen is blank" → "Please check if your computer is plugged in."

Modeling the problem



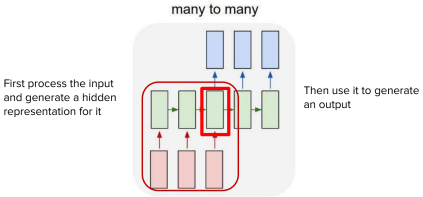
Delayed sequence to sequence

Modeling the problem



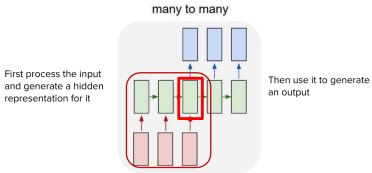
Delayed sequence to sequence

Modeling the problem



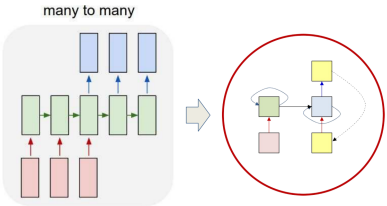
Delayed sequence to sequence

Modeling the problem



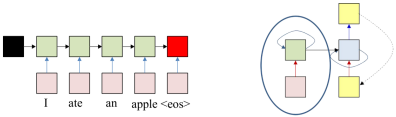
Problem: Each word that is output depends only on current hidden state, and not on previous outputs

Modeling the problem



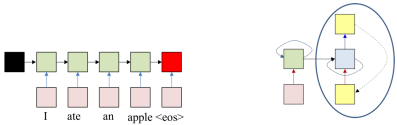
- *Delayed* Sequence to sequence
  - Delayed *self-referencing* sequence-to-sequence

The “simple” translation model



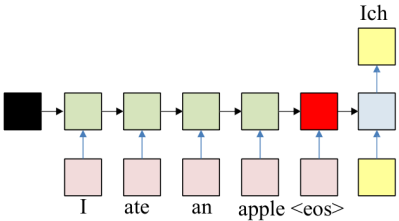
- The input sequence feeds into a recurrent structure
- The input sequence is terminated by an explicit <eos> symbol
  - The hidden activation at the <eos> “stores” all information about the sentence

The "simple" translation model



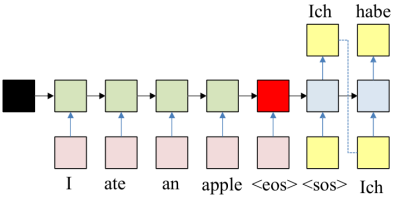
- The input sequence feeds into a recurrent structure
- The input sequence is terminated by an explicit <eos> symbol
  - The hidden activation at the <eos> "stores" all information about the sentence
- Subsequently a second RNN uses the hidden activation as initial state, and <sos> as initial symbol, to produce a sequence of outputs
  - The output at each time becomes the input at the next time
  - Output production continues until an <eos> is produced

The "simple" translation model



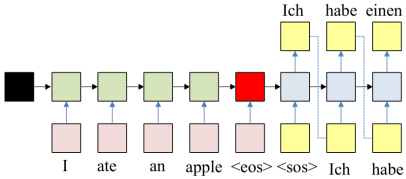
- The input sequence feeds into a recurrent structure
- The input sequence is terminated by an explicit <eos> symbol
  - The hidden activation at the <eos> "stores" all information about the sentence
- Subsequently a second RNN uses the hidden activation as initial state to produce a sequence of outputs
  - The output at each time becomes the input at the next time
  - Output production continues until an <eos> is produced

The "simple" translation model



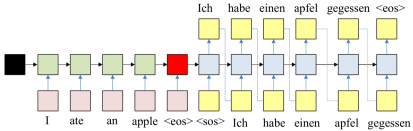
- The input sequence feeds into a recurrent structure
- The input sequence is terminated by an explicit <eos> symbol
  - The hidden activation at the <eos> "stores" all information about the sentence
- Subsequently a second RNN uses the hidden activation as initial state to produce a sequence of outputs
  - The output at each time becomes the input at the next time
  - Output production continues until an <eos> is produced

The "simple" translation model



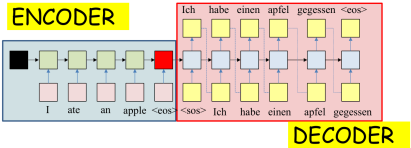
- The input sequence feeds into a recurrent structure
- The input sequence is terminated by an explicit <eos> symbol
  - The hidden activation at the <eos> "stores" all information about the sentence
- Subsequently a second RNN uses the hidden activation as initial state to produce a sequence of outputs
  - The output at each time becomes the input at the next time
  - Output production continues until an <eos> is produced

The “simple” translation model



- The input sequence feeds into a recurrent structure
- The input sequence is terminated by an explicit <eos> symbol
  - The hidden activation at the <eos> “stores” all information about the sentence
- Subsequently a second RNN uses the hidden activation as initial state to produce a sequence of outputs
  - The output at each time becomes the input at the next time
  - Output production continues until an <eos> is produced

The “simple” translation model



- The recurrent structure that extracts the hidden representation from the input sequence is the **encoder**
- The recurrent structure that utilizes this representation to produce the output sequence is the **decoder**

The Decoding Problem

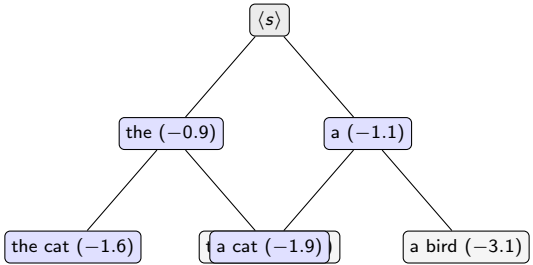
- At each step the decoder outputs a probability distribution over the vocabulary. Decoding is the search for the output sequence with the highest overall probability:

$$\hat{y} = \arg\max_{y} \prod_{t=1}^{T} P(y_t \mid y_{<t}, x)$$

- Searching over all possible sequences is intractable: $|V|^T$ candidates for vocabulary size $|V|$ and length T.
- **Greedy decoding:** take the single most probable token at every step.
- Greedy is cheap but short-sighted: a locally best word can commit the decoder to a worse overall sentence, and there is no way back.

Beam Search

- Keep the k best partial hypotheses (the **beam**) at each step instead of just one.
- At every step, extend each hypothesis with every vocabulary token, score all candidates by cumulative log-probability, and keep the top k.
- $k = 1$ recovers greedy decoding; machine translation typically uses beams of 4 to 10.



Beam of $k = 2$: at each step all extensions are scored and only the two best partial sequences (blue) survive.

Beam Width Trade-offs and Length Normalization

- **Wider beams** cost more compute and give diminishing returns; very wide beams can even hurt quality by surfacing short, generic outputs.
- **Length bias:** summing log-probabilities makes every added token lower the score, so beam search favors short outputs.
- **Length normalization** divides the score by T^{α} (with $\alpha \approx 0.6$ in Wu et al., 2016) so hypotheses of different lengths compete fairly.
- Beam search suits tasks with a well-defined target (translation, summarization). For open-ended generation it tends to produce repetitive, generic text, which is why sampling-based decoding is used there instead.

The Bottleneck Problem

The Bottleneck Problem

Fixed-length context vector = information bottleneck

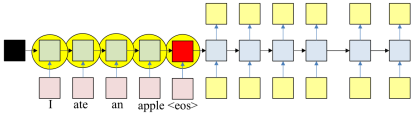
- Encoder must **compress entire input sequence** into a single vector
- Longer or more complex inputs $\rightarrow$ information loss
- Decoder relies solely on that vector to produce outputs

Leads to poor performance on long sentences or tasks requiring high context awareness

A problem with this framework

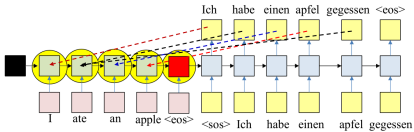
- All the information about the input sequence is embedded into a single vector.
  - The "hidden" node layer at the end of the input sequence.
  - This one node is "overloaded" with information, particularly if the input is long.

A problem with this framework



- In reality: All hidden values carry information.
  - Some of which may be diluted by the time we get to the final state of the encoder.

A problem with this framework



- Every output is related to the input directly.
- Not sufficient to have the encoder hidden state to only the initial state of the decoder.
- Misses the direct relation of the outputs to the inputs.

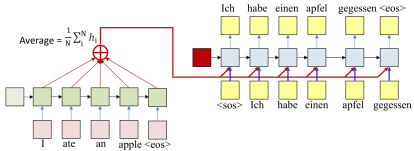
Realization: We Need More Context!

Realization: We Need More Context!

Decoder should have access to all encoder states, not just the final one.

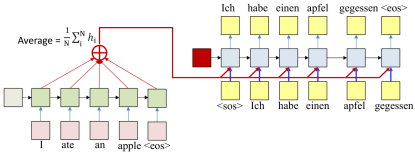
- This inspired the development of the **attention mechanism**.
- Instead of passing only the final state, allow the decoder to *"look back"* at **all input positions**.

Using all input hidden states



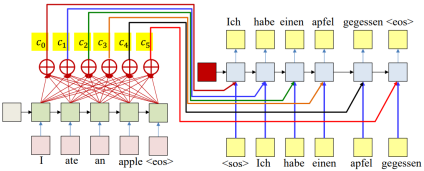
- Simple solution: Compute the average of all encoder hidden states.
- Input this average to every stage of the decoder.
- The initial decoder hidden state is now separate from the encoder and may be a learnable parameter.

Using all input hidden states



- Problem: The average applies the same weight to every input.
- It supplies the same average to every output word.
- In practice, different outputs may be related to different inputs.
  - E.g., “Ich” is most related to “I”, and “habe” and “gegessen” are both most related to “ate”.

Using all input hidden states

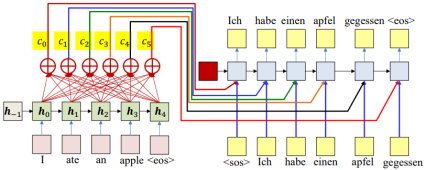


Solution: Use a *different* weighted average for each output word

- The weighted average provided for the kth output word is:

$$c_t = \sum_i w_i(t) h_i$$

Using all input hidden states



$$c_t = \sum_{i=1}^{N} w_i(t) h_i$$

- This solution will work if the weights w_{ki} can somehow be made to "focus" on the right input word
  - E.g., when predicting the word "apfel", $w_3(4)$, the weight for "apple" must be high while the rest must be low
- How do we generate such weights??

Introduction to Attention Mechanism

Introduction to Attention Mechanism

Core Idea: Let the decoder **focus on different parts of the input** sequence at each step of decoding.

- At each decoding step, compute a **weighted sum** over all encoder hidden states.
- Weights reflect **relevance** of each input word to the current output word.

"Soft search" over inputs → more context-awareness.

Mathematical Formulation of Attention

1. **Alignment Score:**

$$e_{t,s} = \text{score}(h_t^{\text{dec}}, h_s^{\text{enc}})$$

2. **Attention Weights (Softmax):**

$$\alpha_{t,s} = \frac{\exp(e_{t,s})}{\sum_{s'} \exp(e_{t,s'})}$$

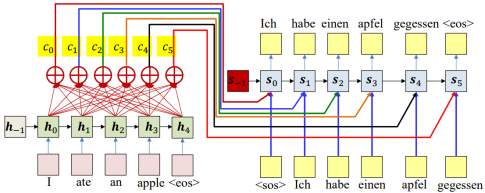
3. **Context Vector:**

$$c_t = \sum_s \alpha_{t,s} h_s^{\text{enc}}$$

4. **Decoder Input:**

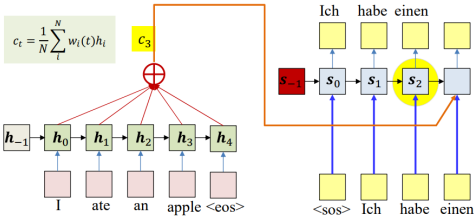
$$y_t = \text{Decoder}(y_{t-1}, h_{t-1}^{\text{dec}}, c_t)$$

Attention Models



- **Attention weights:** The weights are dynamically computed as functions of decoder state.
- **Expectation:** if the model is well-trained, this will automatically "highlight" the correct input.
- But how are these computed?

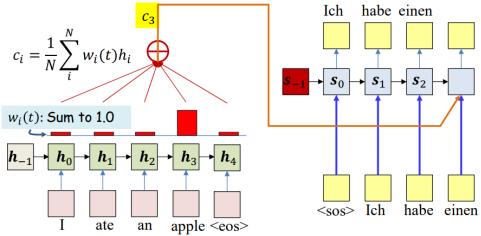
Attention weights at time t



- The "attention" weights $w_i(t)$ must be computed from available information at time t.
- The primary information is s_{t-1} (the state at time $t-1$).
- Also, the input word at time t, but generally not used for simplicity.

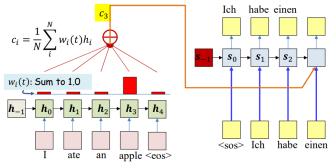
$$c_t = \sum_{i=1}^{N} w_i(t) h_i$$

Requirement on attention weights



- The weights must be positive and sum to 1.0 (i.e., be a distribution).
- Ideally, they must be high for the most relevant inputs for the *i*-th output and low elsewhere.
- **Solution: A two step weight computation**
  - First compute raw weights (which could be +ve or –ve).
  - Then softmax them to convert them to a distribution.

Requirement on attention weights



- The weights $w_i(t)$ must be positive and sum to 1.0
  - I.e. be a distribution
  - Ideally, they must be high for the most relevant inputs for the ith output and low elsewhere
- Solution: A two step weight computation
  - First compute *raw* weights (which could be +ve or –ve)
  - Then softmax them to convert them to a distribution

$$e_i(t) = g(\boldsymbol{h}_i, \boldsymbol{s}_{t-1})$$

$$w_i(t) = \frac{\exp(e_i(t))}{\sum_j \exp(e_j(t))}$$

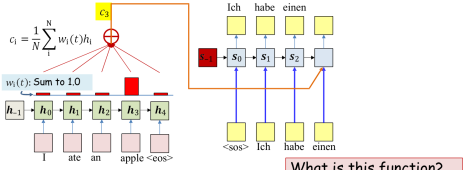
Quiz: Attention Framework

1. The attention framework computes a different "context" vector at each output step?
2. The context vector is chosen as the hidden representation of the word with the highest weight?
3. The weight is a function of the encoder representation and the most recent decoder state?

Quiz: Answer

1. The attention framework computes a different "context" vector at each output step? **True**
2. The context vector is chosen as the hidden representation of the word with the highest weight? **False**
3. The weight is a function of the encoder representation and the most recent decoder state? **True**

Requirement on attention weights



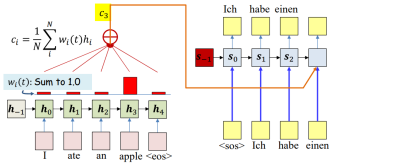
- The weights $w_i(t)$ must be positive and sum to 1.0
  - i.e. be a distribution
  - Ideally, they must be high for the most relevant inputs for the ith output and low elsewhere
- Solution: A two step weight computation
  - First compute *raw* weights (which could be +ve or –ve)
  - Then softmax them to convert them to a distribution

What is this function?

$$e_i(t) = g(\boldsymbol{h}_i, s_{t-1})$$

$$w_i(t) = \frac{\exp(e_i(t))}{\sum_j \exp(e_j(t))}$$

Attention weights



- Typical options for $g()$ (**variables in red must learned**)

$g(\boldsymbol{h}_i, \boldsymbol{s}_{t-1}) = \boldsymbol{h}_i^T \boldsymbol{s}_{t-1}$

$g(\boldsymbol{h}_i, \boldsymbol{s}_{t-1}) = \boldsymbol{h}_i^T \boldsymbol{W}_g \boldsymbol{s}_{t-1}$

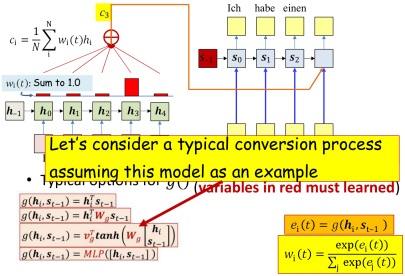
$g(\boldsymbol{h}_i, \boldsymbol{s}_{t-1}) = \boldsymbol{v}_g^T \boldsymbol{tanh}\left(\boldsymbol{W}_g \begin{bmatrix} \boldsymbol{h}_i \\ \boldsymbol{s}_{t-1} \end{bmatrix}\right)$

$g(\boldsymbol{h}_i, \boldsymbol{s}_{t-1}) = \boldsymbol{MLP}([\boldsymbol{h}_i, \boldsymbol{s}_{t-1}])$

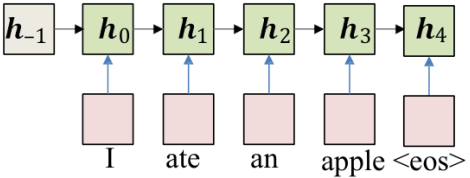
$e_i(t) = g(\boldsymbol{h}_i, \boldsymbol{s}_{t-1})$

$w_i(t) = \frac{\exp(e_i(t))}{\sum_j \exp(e_j(t))}$

Attention weights

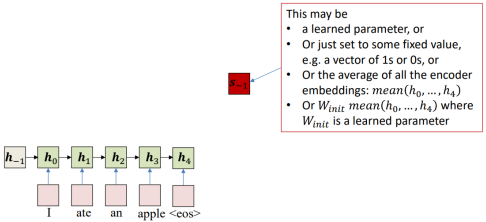


Converting an input: Inference



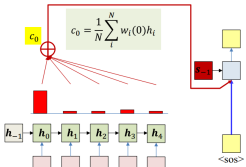
Pass the input through the encoder to produce hidden representations h_i

Converting an input: Inference



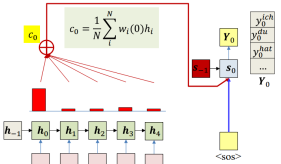
Pass the input through the encoder to produce hidden representations h_i

Converting an input: Inference



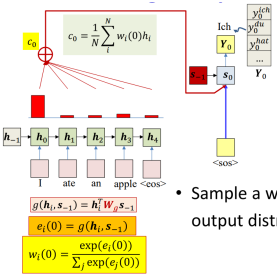
- Compute the attention weights $w_i(0)$ for the first output using s_{-1}
  - Will be a distribution over the input words
- Compute "context" c_0
  - Weighted sum of input word hidden states
- Input c_0 and <sos> to the decoder at time 0
  - <sos> because we are starting a new sequence
  - In practice we will enter the *embedding* of <sos>

Converting an input: Inference



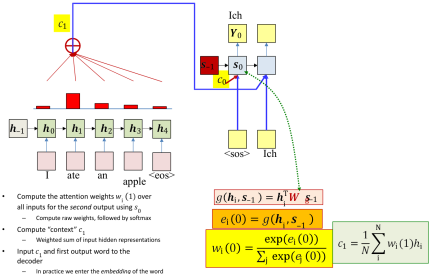
- The decoder computes
  - s_0
  - A probability distribution over the output vocabulary
    - Output of softmax output layer

Converting an input: Inference



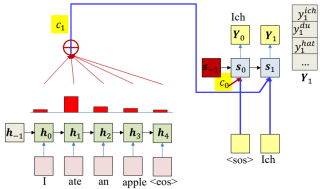
- Sample a word from the output distribution

Converting an input: Inference



- Compute the attention weights $w_i(1)$ over all inputs for the *second* output using s_0
  - Compute raw weights, followed by softmax
- Compute "context" c_1
  - Weighted sum of input hidden representations
- Input c_1 and first output word to the decoder
  - In practice we enter the *embedding* of the word

Converting an input: Inference



- The decoder computes
  - s_1
  - A probability distribution over the output vocabulary

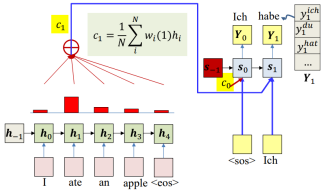
$$g(\boldsymbol{h}_i, \boldsymbol{s}_0) = \boldsymbol{h}_i^T \boldsymbol{W}_g \boldsymbol{s}_0$$

$$e_i(1) = g(\boldsymbol{h}_i, \boldsymbol{s}_0)$$

$$w_i(1) = \frac{\exp(e_i(1))}{\sum_j \exp(e_j(1))}$$

$$c_1 = \frac{1}{N} \sum_i^N w_i(1) h_i$$

Converting an input: Inference



- Sample the second word from the output distribution

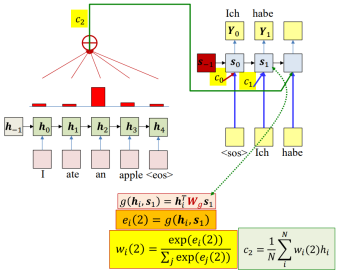
$$g(\boldsymbol{h}_i, \boldsymbol{s}_0) = \boldsymbol{h}_i^T \boldsymbol{W}_g \boldsymbol{s}_0$$

$$e_i(1) = g(\boldsymbol{h}_i, \boldsymbol{s}_0)$$

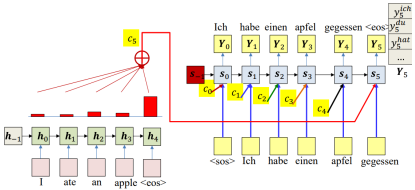
$$w_i(1) = \frac{\exp(e_i(1))}{\sum_j \exp(e_j(1))}$$

$$c_1 = \frac{1}{N}\sum_i^N w_i(1)h_i$$

Converting an input: Inference



Converting an input: Inference



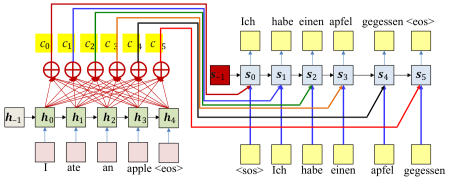
Continue this process until <eos> is drawn

$$e_i(5) = g(h_i, s_4)$$

$$w_i(5) = \frac{\exp(e_i(5))}{\sum_j \exp(e_j(5))}$$

$$c_5 = \frac{1}{N}\sum_i^N w_i(5) h_i$$

Attention-based Decoding

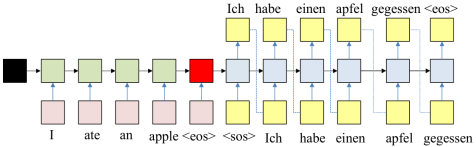


$$e_i(t) = g(\boldsymbol{h}_i, \boldsymbol{s}_{t-1})$$

$$w_i(t) = \frac{\exp(e_i(t))}{\sum_j \exp(e_j(t))}$$

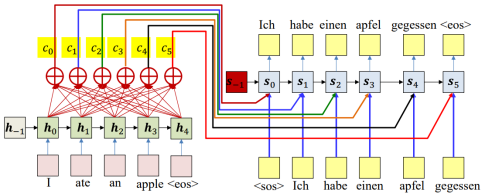
$$c_t = \frac{1}{N}\sum_{i}^{N} w_i(t) h_i$$

Recap: Sequence to Sequence Models



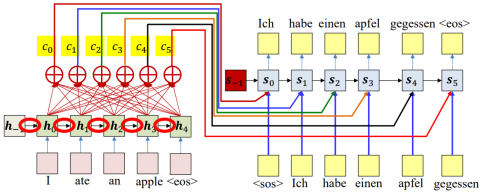
- The input sequence feeds into a recurrent structure
- The input sequence is terminated by an explicit <eos> symbol
  - The hidden activation at the <eos> "stores" all information about the sentence
- Subsequently a second RNN uses the hidden activation as initial state to produce a sequence of outputs

Recap: Attention Models



- Encoder recurrently produces hidden representations of input word sequence
- Decoder recurrently generates output word sequence
  - For each output word the decoder uses a weighted average of the hidden input representations as input "context", along with the recurrent hidden state and the previous output word

Recap: Attention Models



- **Problem:** Because of the recurrence, the hidden representation for any word is also influenced by *all* preceding words
  - The decoder is actually paying attention to the sequence, and not just the word

Impact of Attention

- Improves translation accuracy
- Helps handle **longer sequences**
- Interpretable: shows what the model is focusing on
- Forms the foundation of **Transformer models**

Led to development of:

- Bahdanau Attention (Additive, 2014)
- Luong Attention (Multiplicative, 2015)
- Self-Attention and Transformers (2017+)

Seq2Seq vs Seq2Seq + Attention

Aspect	**Basic Seq2Seq**	**With Attention**
Memory	Single vector (fixed)	Multiple encoder states (dynamic)
Long Sequences	Poor performance	Good performance
Interpretability	Low	High (via attention weights)
Use Cases	Short/medium sequences	Longer, complex tasks

Applications of Seq2Seq + Attention

- Machine Translation
- Text Summarization
- Chatbots and Dialog Systems
- Speech Recognition
- Question Answering
- Video Captioning
- DNA Sequence Modeling

BLEU (Bilingual Evaluation Understudy)

- BLEU is an n-gram overlap metric commonly used to evaluate text generation tasks such as machine translation. It compares generated text against reference text using precision of n-grams.
- Formula:

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^{N} w_n \log p_n\right)$$

where:

- p_n = precision of n-grams
- w_n = weight for each n-gram order (usually uniform)
- BP = brevity penalty to penalize short hypotheses

BLEU (Bilingual Evaluation Understudy) (cont.)

- How it works:
  - BLEU evaluates how many n-grams in the hypothesis match the reference.
  - Missing or misordered words reduce the score.
- Example:
  - Reference: "the cat is on the mat"
  - Hypothesis: "the cat on mat"
  - Unigram precision $= 4/4 = 1.0$
  - Bigram precision $= 1/3 \approx 0.33$
  - BLEU penalizes missing words via BP.
- Why it matters:
  - Easy to compute and widely adopted for automatic evaluation of generated text.

ROUGE (Recall-Oriented Understudy for Gisting Evaluation)

- ROUGE is a recall-focused metric widely used in text summarization to evaluate how much of the reference content is captured by the generated text.
- Common variants:
  - **ROUGE-N:** recall of n-grams.
  - **ROUGE-L:** based on the longest common subsequence.
- Formula (ROUGE-N):

$$\text{ROUGE-N} = \frac{\sum_{\text{n-gram} \in \text{Ref}} \min(\text{count}_{\text{hyp}}, \text{count}_{\text{ref}})}{\sum_{\text{n-gram} \in \text{Ref}} \text{count}_{\text{ref}}}$$

ROUGE (Recall-Oriented Understudy for Gisting Evaluation) (cont.)

- Example:
  - Reference: "the cat sat on the mat"
  - Hypothesis: "the cat sat on mat"
  - ROUGE-1 recall $= 5/6 \approx 83\%$
- Why it matters:
  - Good for checking content coverage in summarization tasks.

METEOR & TER (Other Evaluation Metrics)

- **METEOR:** Measures text generation quality considering not just exact word matches but also stemming, synonyms, and paraphrases. Tends to correlate better with human judgment than BLEU.
- **TER (Translation Edit Rate):** Counts the minimum number of edits (insertions, deletions, substitutions, and shifts) needed to match the reference text. Lower TER indicates better performance.
- Why these metrics matter:
  - Often used alongside BLEU and ROUGE to get a more complete evaluation of generated text quality.

Automated Metrics vs Human Evaluation

- **Automated Metrics** (e.g., BLEU, ROUGE):
  - **Positive:** Cheap, fast, and reproducible.
  - **Negative:** Limited depth; may not capture fluency, naturalness, or true meaning.
- **Human Evaluation:**
  - **Positive:** Captures fluency, naturalness, and semantic meaning.
  - **Negative:** Expensive, slow, and less reproducible.
- Why both matter:
  - Automated metrics allow rapid iteration and benchmarking.
  - Human evaluation ensures quality aligns with real-world perception.
  - Best practice: combine both depending on the task.

Example: BLEU vs ROUGE in Practice

- Reference: “The cat sat on the mat.”
- Hypotheses:
  - Hypothesis A: “The cat is on the mat.”
  - Hypothesis B: “A feline rested on the rug.”
- Metric evaluation:
  - **BLEU:** A ¿ B (closer word overlap with reference)
  - **ROUGE:** A ¿ B (better recall of reference n-grams)
  - Hypothesis B is semantically correct, yet both metrics punish it.
- Takeaway:
  - Different metrics capture different aspects of quality.
  - Word overlap metrics may disagree with human or semantic judgment.

Limitations

- Still sequential — **not fully parallelizable**
- Attention adds computational cost
- Hard to interpret in large-scale models
- Might struggle with very long-range dependencies in huge contexts

Future Directions

- **Transformers**: Fully attention-based, no recurrence
- **Efficient Attention Models**: Longformer, Reformer, Linformer
- **Multimodal Attention**: Vision + Text
- **Memory-Augmented Models**
- **Structured Attention**: Parses, syntax, and alignment bias

Attention paved the way for GPT, BERT, and LLMs

Summary

- Seq2Seq enables mapping input to output sequences of variable lengths
- **Bottleneck:** Fixed-size context vector limits learning capacity
- **Attention:** Improves performance by giving **adaptive access** to input states
- Attention $\rightarrow$ Transformer $\rightarrow$ LLMs
- Still evolving: From additive attention to self-attention and beyond

References

These slides have been adapted from:

- Younes Mourri & Lukasz Kaiser, Natural Language Processing Specialization, DeepLearning.AI
- Bhiksha Raj & Rita Singh, 11-785 Introduction to Deep Learning, CMU

References

Foundational Papers:

- Sutskever et al. (2014). *Sequence to Sequence Learning with Neural Networks*. NeurIPS.
- Bahdanau et al. (2014). *Neural Machine Translation by Jointly Learning to Align and Translate*.
- Luong et al. (2015). *Effective Approaches to Attention-based Neural Machine Translation*.
- Vaswani et al. (2017). *Attention Is All You Need* (Transformer).
- Chan et al. (2016). *Listen, Attend and Spell* (Speech recognition).

References

Courses & Tutorials:

- Stanford CS224n: Lecture 9 (Attention)
- DeepLearning.ai NLP Specialization (Coursera)
- Harvard NLP Annotated Transformer: http://nlp.seas.harvard.edu/2018/04/03/attention.html
- Jay Alammar's blog: "The Illustrated Transformer"